SAE3.02 Programmation Client/Serveur

Boyer Maxence
RT2

Sommaire

Introduction_____ page 3
Analyse du cahier des charges_____ page 4
Contraintes techniques
Fonctionnalités à implémenter
Approche retenue
Développement progressif_____ page 6
Mode TCP et envoie de données complexes
Multithreading pour gérer plusieurs Clients
Serveur Multi-services et droits d'accès
Tests et validation_____ page 11
Implémentation et Validation du protocole UDP
Architecture service pour le fonctionnement en arrière-plan
Améliorations apportées_____ page 14
Refonte de l'interface graphique
Amélioration Ergonomiques et fonctionnelles
Architecture Service pour un Serveur en Arrière-plan
Conclusion_____ page 22
Annexe dépôt du code_____page 24

Introduction

L'objectif de ce projet était de créer une application Android bidirectionnelle (client/serveur) utilisant les protocoles TCP et UDP pour échanger des données sur un réseau local.

J'ai travaillé uniquement avec Java et XML sous Android Studio, ce qui m'a obligé à gérer moi-même toute la partie réseau sans utiliser de bibliothèques externes modernes. Les principales contraintes étaient :
- Permettre à l'application de basculer entre mode client et serveur
- Gérer plusieurs connexions simultanées avec du multithreading
- Implémenter un système multi-services où le serveur répond différemment selon le type de requête
- Ajouter une couche de sécurité avec des droits d'accès

Dans ce document, je vais expliquer comment j'ai organisé mon code, les choix techniques que j'ai faits, et surtout montrer les résultats de mes tests progressifs.

ANALYSE DU CAHIER DES CHARGES

Avant de coder, j'ai analysé le sujet pour identifier les fonctionnalités à développer et anticiper les difficultés.

2.1 Contraintes techniques

Le projet devait être développé en Java et XML sous Android Studio. Cette contrainte m'a obligé à manipuler directement les classes Socket et ServerSocket sans bibliothèques modernes qui simplifient le travail.

L'application devait fonctionner en mode client ou serveur. J'ai donc dû concevoir deux logiques opposées dans une même application : le serveur écoute et attend, le client se connecte. Pour gérer les modes client et serveur, j'ai opté pour une unique MainActivity dont l'interface se reconfigure dynamiquement grâce à un SwitchCompat. Cette approche centralise la logique de l'interface et évite la duplication de code entre deux activités distinctes.

2.2 Fonctionnalités à implémenter

Protocoles TCP et UDP
Les deux protocoles devaient être supportés. TCP pour les connexions fiables, UDP pour les échanges rapides sans garantie. Ça impliquait deux implémentations différentes avec Socket (TCP) et DatagramSocket (UDP).

Structures complexes
Au-delà du simple texte, il fallait envoyer des objets Java sérialisés et des tableaux d'octets. J'ai compris qu'il faudrait utiliser ObjectOutputStream pour la sérialisation.

Multithreading serveur
Le serveur devait gérer plusieurs clients simultanément. Ça voulait dire créer un thread par client pour éviter de bloquer le serveur.

Multi-services
Le client devait pouvoir demander différents services (echo, calcul, etc.). Le serveur devait analyser la demande et router vers le bon traitement, un peu comme un switch selon le type de requête.

Droits d'accès
Certains services devaient être réservés à certains utilisateurs. J'ai opté pour un système simple : login/mot de passe en local avec vérification avant chaque service.

2.3 Approche retenue

J'ai décidé de développer progressivement plutôt que tout d'un coup :

1. TCP simple (texte uniquement)
2. Ajout UDP
3. Multithreading
4. Multi-services
5. Droits d'accès
6. Améliorations personnelles

Cette méthode me permettait de tester chaque fonctionnalité avant d'ajouter la suivante. Pour les initiatives personnelles, j'ai prévu d'améliorer l'interface et d'ajouter un système de logs.

Développement Progressif

Pour aborder ce projet complexe, j'ai adopté une approche itérative en ajoutant les fonctionnalités les unes après les autres. Cette méthode m'a permis de tester et de valider chaque brique de l'application avant de passer à la suivante, garantissant ainsi une base stable à chaque étape.

3.1 Mode TCP et Envoi de Données Complexes

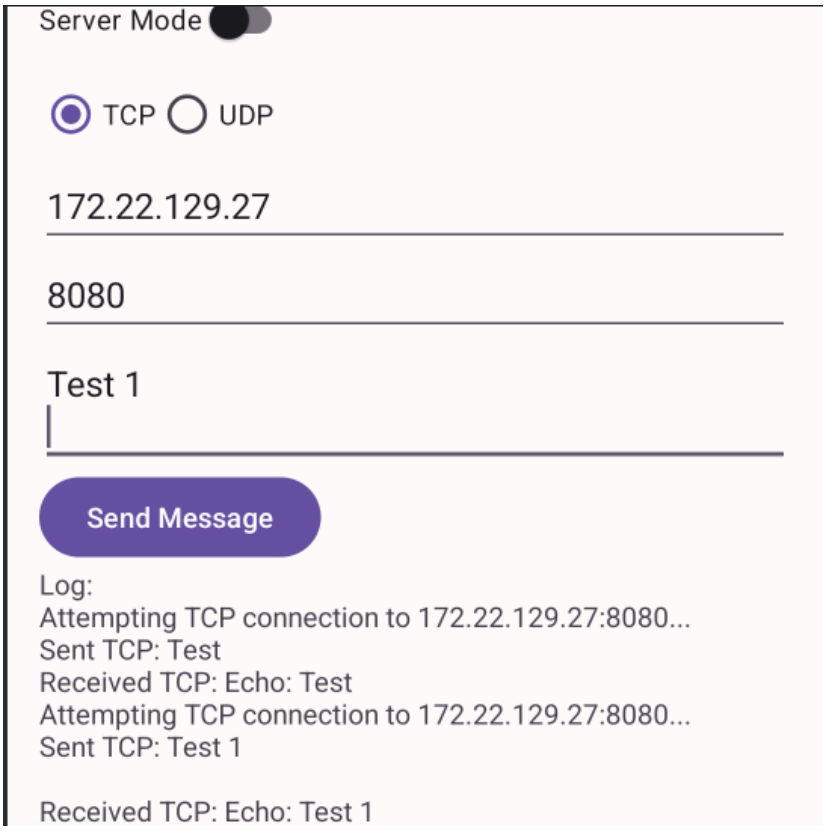
La première étape a été de mettre en place la communication la plus simple et la plus fiable : le protocole TCP. L'objectif était de permettre au serveur d'accepter une connexion, et au client d'envoyer un message.

Pour répondre à l'exigence d'envoyer des "structures complexes", j'ai immédiatement créé une classe Message qui implémente l'interface Serializable. Cette classe encapsule toutes les informations nécessaires à une requête : l'identifiant de l'utilisateur, son mot de passe et la commande à exécuter.

```
Java

// Fichier : Message.java

public class Message implements Serializable {

    private static final long serialVersionUID = 7526472295622776147L;



    private final String username;

    private final String password;

    private final String command;


    // Constructeur et getters...

}
```

Côté réseau, j'ai utilisé ObjectOutputStream pour envoyer l'objet Message entier à travers le Socket TCP. Cela simplifie grandement la communication, car Java gère lui-même la conversion de l'objet en un flux d'octets.

Les premiers tests ont été concluants. Comme le montre la capture d'écran ci-dessous, le client parvient à se connecter au serveur et à envoyer un premier type de commande, validant ainsi la connexion TCP initiale.



3.2 Multithreading pour Gérer Plusieurs Clients

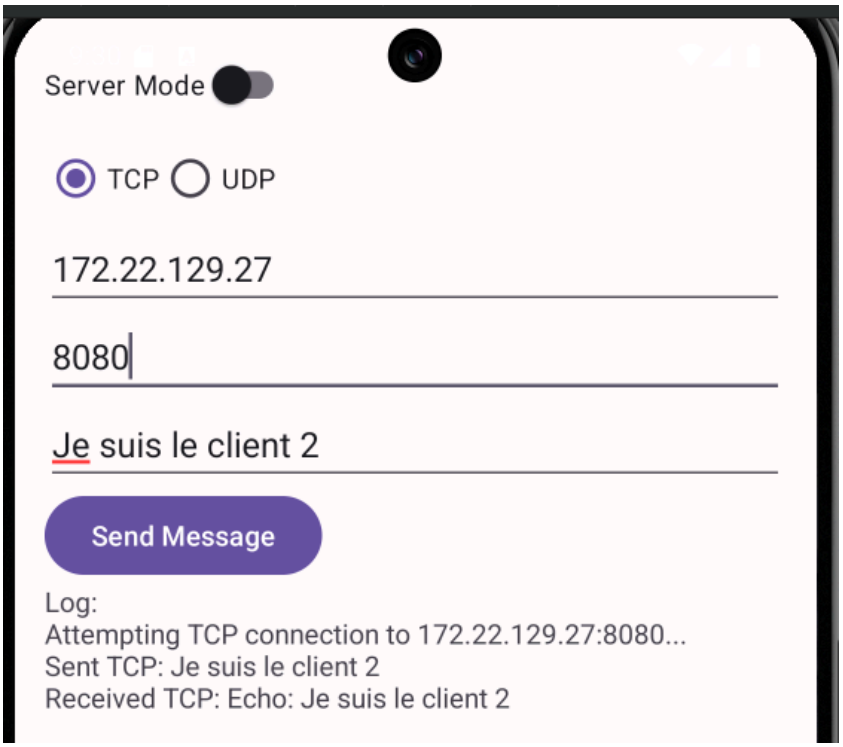
Une fois la connexion unique fonctionnelle, il était crucial de permettre au serveur de gérer plusieurs clients en même temps sans qu'ils se bloquent mutuellement. Si un seul thread était utilisé, le serveur serait incapable de répondre à un deuxième client tant qu'il n'aurait pas fini avec le premier.

Pour résoudre ce problème, j'ai utilisé un ExecutorService, plus précisément un Executors.newCachedThreadPool(). C'est une approche moderne et efficace qui gère un pool de threads. Chaque fois qu'un nouveau client se connecte (serverSocket.accept()), je délègue le traitement de ce client à un nouveau thread du pool.

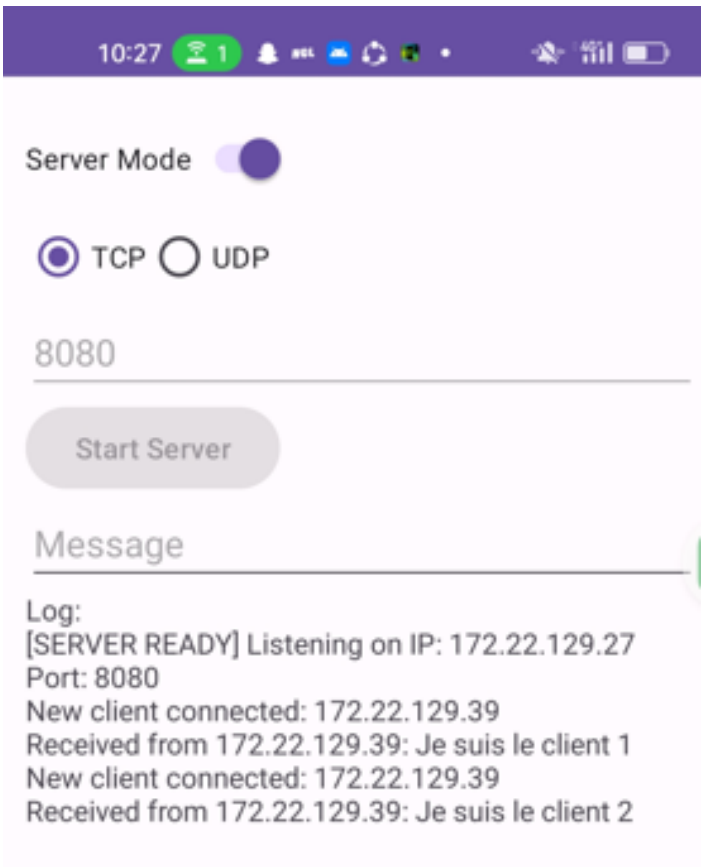
```
// Extrait de ServerService.java

// Boucle principale du serveur TCP
while (isServerRunning) {
    Socket clientSocket = serverSocket.accept();
    logMessage("TCP Client connected: " +
clientSocket.getInetAddress().getHostAddress());
    // On délègue le client à un thread séparé
    executorService.execute(() -> handleTCPClient(clientSocket));
}
```

Cette architecture permet au serveur de rester réactif et d'accepter de nouvelles connexions immédiatement, tandis que les communications existantes se poursuivent en parallèle.

Le test, visible sur les deux captures d'écrans ci-dessous, nous montre deux clients différents connectés et communiquant avec le serveur en même temps, prouvant le bon fonctionnement du multithreading.



Et du côté du serveur:



3.3 Serveur Multi-Services et Droits d'Accès

Pour que le serveur puisse offrir différents services et gérer les droits, j'ai centralisé le traitement des requêtes dans une méthode processClientRequest. Cette méthode agit comme un aiguillage :

1. Authentification : Elle vérifie d'abord si le nom d'utilisateur et le mot de passe fournis dans l'objet Message correspondent à une paire valide stockée dans une HashMap. Si ce n'est pas le cas, l'accès est refusé.
2. Analyse de la commande : Si l'authentification réussit, la méthode analyse la chaîne de commande (par exemple, DATE, ECHO:Bonjour, CALCUL:10+5).
3. Aiguillage (switch) : Un bloc switch redirige la requête vers la logique appropriée.

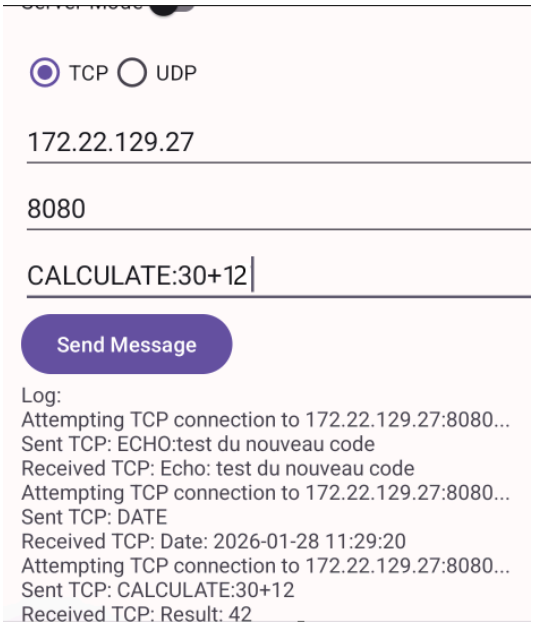
```
// Extrait de la méthode processClientRequest dans ServerService.java
private String processClientRequest(Message message) {
    // ... vérification des identifiants ...

    switch (commandName) {
        case "DATE":
```

```
        // ... renvoyer la date ...
    case "ECHO":
        // ... renvoyer l'argument ...
    case "CALCUL":
        return calculate(argument);
    default:
        return "Error: Unknown command";
  }
}
```

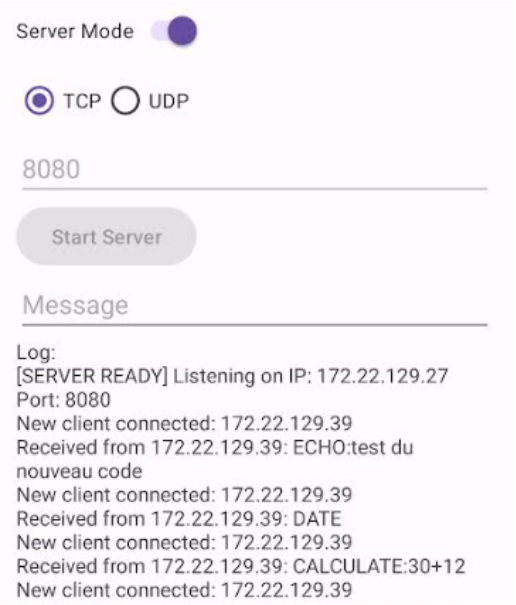
Cette structure est très flexible et permet d'ajouter facilement de nouveaux services à l'avenir sans modifier le reste du code.

Les tests, visibles ci-dessous, montrent que le client reçoit des réponses différentes en fonction des commandes envoyées (DATE, ECHO, CALCUL), ce qui valide le système multi-services et la gestion des droits.

Sur le client



Sur le serveur



4. Tests et Validation

Après le développement progressif, une phase de tests finaux a été menée pour valider l'ensemble des fonctionnalités, y compris le protocole UDP et l'architecture Service.

4.1 Implémentation et Validation du Protocole UDP

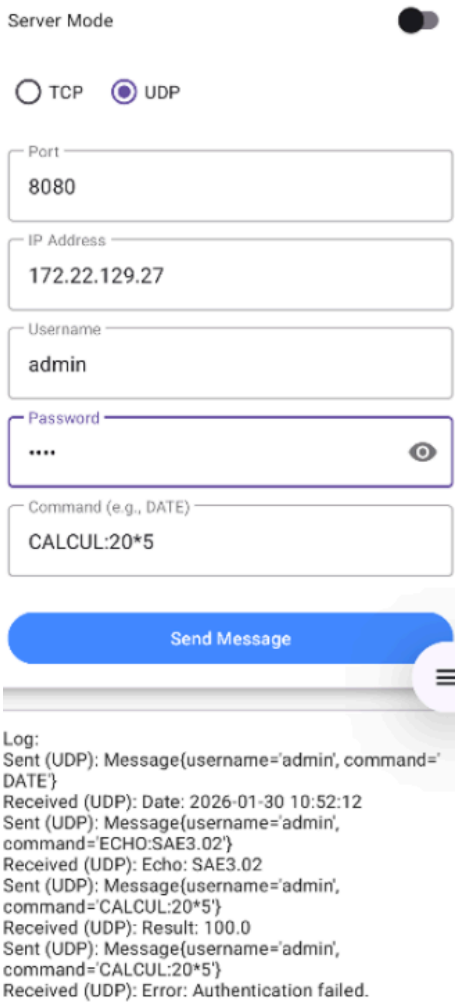
L'ajout du protocole UDP a nécessité une approche différente, car il ne fonctionne pas avec des flux de connexion, mais avec des paquets de données (DatagramPacket).

- Côté Serveur : Le serveur UDP écoute sur un DatagramSocket et attend de recevoir des paquets.
- Côté Client : Le client UDP sérialise l'objet Message dans un tableau d'octets (byte[]), l'enveloppe dans un DatagramPacket et l'envoie à l'adresse du serveur.

La mise en œuvre du protocole UDP a présenté un défi de débogage intéressant. Lors des premiers tests, la communication échouait systématiquement avec une erreur de timeout. Après une analyse méthodique, j'ai découvert que le problème ne venait pas du code lui-même, mais de la manière dont le réseau virtuel des émulateurs Android gère les paquets UDP, contrairement au TCP. La validation finale sur deux appareils physiques a permis de prouver que l'implémentation était correcte. Cette

expérience a été très instructive sur l'importance des conditions de test en réseau.

La capture d'écran ci-dessous montre une session client complète en mode UDP. Toutes les commandes (DATE, ECHO, CALCUL) sont envoyées et leurs réponses sont reçues correctement, ce qui prouve la bonne implémentation du protocole.



4.2 Architecture Service pour le Fonctionnement en Arrière-Plan

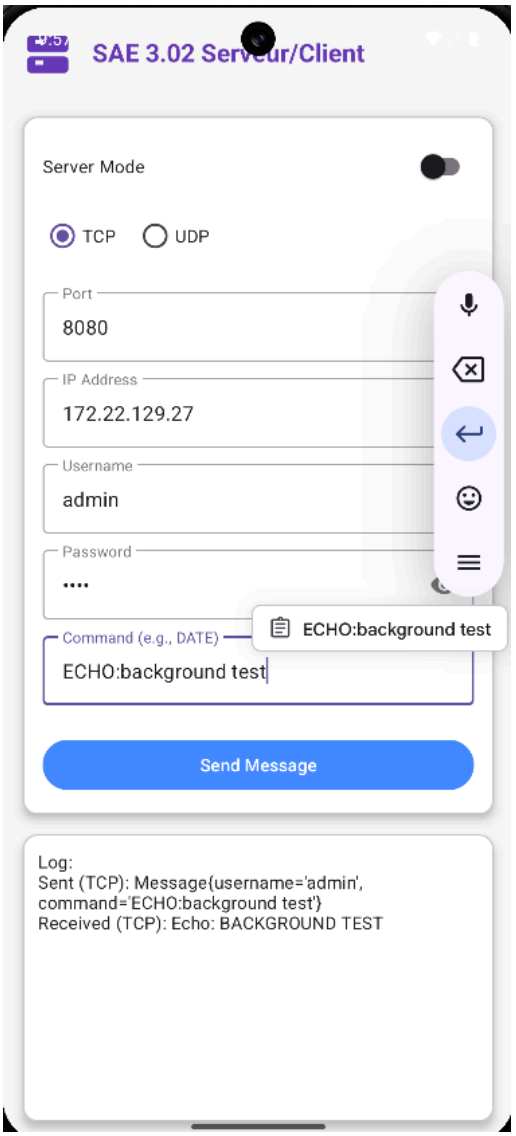
Pour répondre à l'attente d'une application professionnelle et robuste, j'ai déplacé toute la logique du serveur dans un Service Android. C'est l'amélioration la plus significative du projet.

- ServerService.java : Cette nouvelle classe contient désormais toute la logique réseau du serveur.

- MainActivity.java : L'activité est devenue une simple "télécommande". Elle envoie des Intent au service pour le démarrer ou l'arrêter, et elle reçoit des mises à jour (logs, état du serveur) via un BroadcastReceiver.

Cette architecture permet au serveur de continuer à fonctionner même si l'utilisateur quitte l'application.

Le test, visible ci-dessous, en est la preuve. Le serveur a été démarré, puis l'application a été mise en arrière-plan. Un client a ensuite envoyé la commande ECHO:background test. La capture d'écran montre que le serveur, bien qu'invisible, a bien reçu et traité la requête, confirmant son fonctionnement autonome.



5. Améliorations Apportées

Au-delà des exigences initiales du cahier des charges, j'ai pris plusieurs initiatives pour améliorer la qualité, la robustesse et l'ergonomie de l'application. Ces améliorations ont transformé un projet fonctionnel en une application complète et soignée.

5.1 Refonte de l'Interface Graphique

L'interface initiale était fonctionnelle mais peu intuitive. Pour la rendre plus professionnelle et agréable à utiliser, je l'ai entièrement reconstruite en utilisant les composants modernes de Material Design.

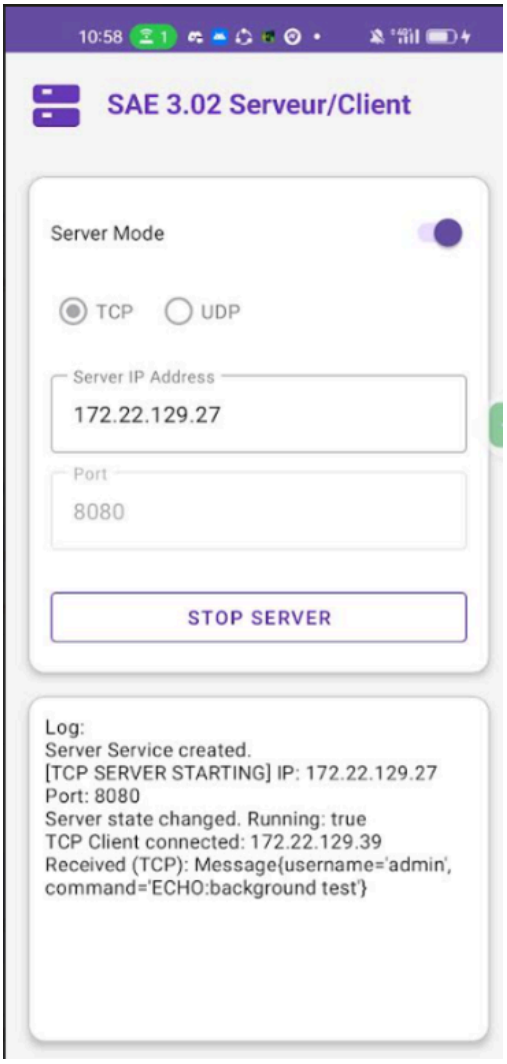
J'ai remplacé la structure LinearLayout de base par un ConstraintLayout plus flexible. Les groupes de champs (configuration, authentification, etc.) ont été encapsulés dans des MaterialCardView pour créer une séparation visuelle claire. De plus, chaque EditText a été enveloppé dans un TextInputLayout, ce qui apporte des libellés flottants, des compteurs et des icônes (comme l'œil pour afficher/cacher le mot de passe), améliorant grandement l'expérience utilisateur. Enfin, j'ai créé une palette de couleurs et un logo pour donner une identité visuelle au projet.

```
<!-- Extrait de activity_main.xml montrant la nouvelle structure -->
<com.google.android.material.card.MaterialCardView
    android:id="@+id/config_card"
    ...>

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
```

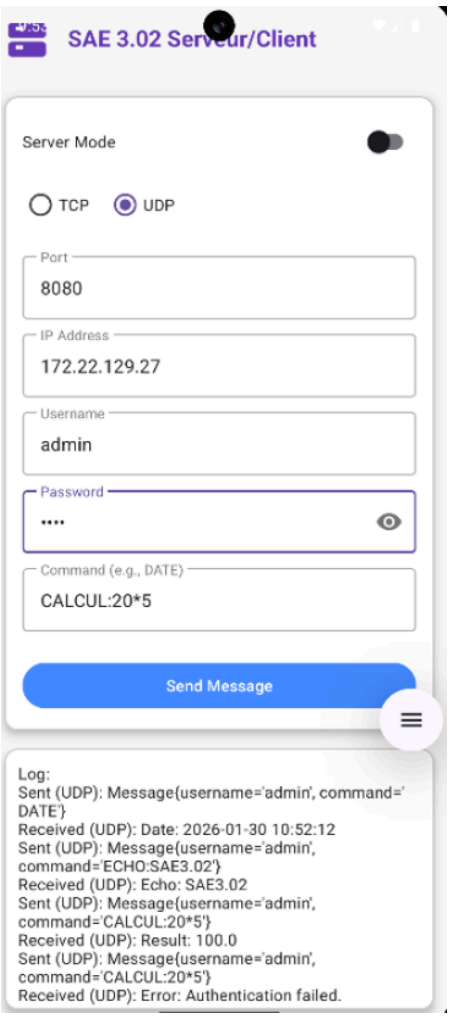
```
        android:padding="16dp">

        <com.google.android.material.textfield.TextInputLayout
            android:id="@+id/layoutPort"

style="@style/Widget.MaterialComponents.TextInputLayout.Outlin
edBox"
            android:hint="Port">

            <com.google.android.material.textfield.TextInputEditText
                android:id="@+id/editTextPort"
                android:inputType="number" />

        </com.google.android.material.textfield.TextInputLayout>

        <!-- ... autres champs de saisie ... -->

    </LinearLayout>
</com.google.android.material.card.MaterialCardView>
```

Le résultat est visible sur n'importe quelle capture d'écran finale. En comparant les premiers tests avec les derniers, on voit clairement la montée en qualité de l'interface, la rendant plus propre et plus facile à comprendre.

côté serveur


10:58
SAE 3.02 Serveur/Client
Server Mode
TCP
UDP
Server IP Address
172.22.129.27
Port
8080
STOP SERVER
Log:
Server Service created.
[TCP SERVER STARTING] IP: 172.22.129.27
Port: 8080
Server state changed. Running: true
TCP Client connected: 172.22.129.39
Received (TCP): Message{username='admin',
command='ECHO:background test'}

Côté client



5.2 Améliorations Ergonomiques et Fonctionnelles

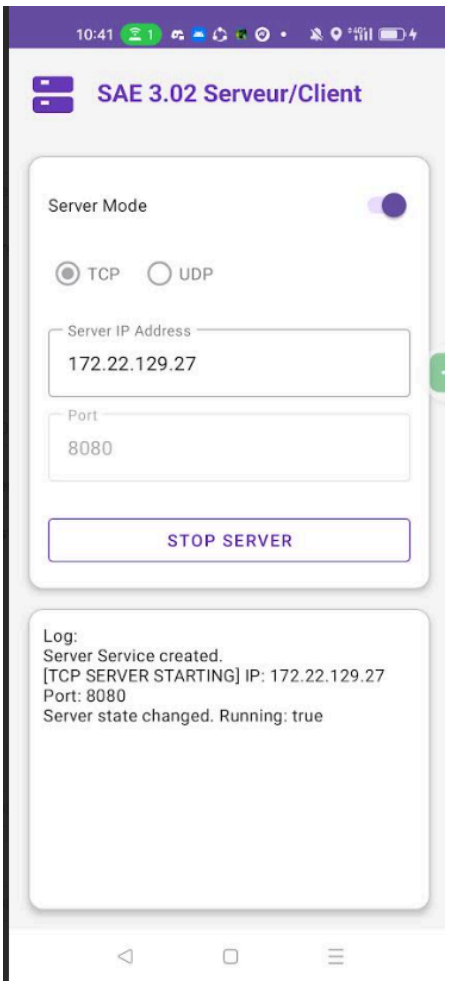
Pour faciliter l'utilisation, j'ai ajouté plusieurs fonctionnalités qui n'étaient pas demandées :

1. Un bouton "Stop Server" : Essentiel pour permettre un contrôle total du serveur sans avoir à redémarrer l'application.
2. Affichage de l'IP du serveur : Au lieu de devoir chercher l'IP dans les logs, un champ dédié et non modifiable apparaît dès que le serveur est démarré. C'est un gain de temps considérable pour les tests.
3. Une commande CALCUL : Pour enrichir le côté "multi-services", j'ai ajouté une commande qui permet au serveur d'effectuer des calculs simples.

Pour cela, j'ai modifié la méthode updateUI() pour gérer dynamiquement la visibilité des boutons Start/Stop et du champ de l'IP en fonction de l'état du serveur.

```
// Extrait de la méthode updateUI() dans MainActivity.java
private void updateUI() {
    handler.post(() -> {
        boolean isServerRunning = ServerService.isServerRunning;
        if (isServer) {
            if (isServerRunning) {
                buttonStartServer.setVisibility(View.GONE);
                buttonStopServer.setVisibility(View.VISIBLE);
                layoutServerIp.setVisibility(View.VISIBLE);
            } else {
                // ... cacher les éléments ...
            }
        }
        // ...
    });
}
```

La capture ci-dessous serveur TCP.png, nous montre ces améliorations : le bouton "STOP SERVER" est actif et le champ "Server IP Address" affiche l'adresse, rendant l'interface beaucoup plus claire.



5.3 Architecture Service pour un Serveur en Arrière-Plan

C'est l'amélioration la plus importante et la plus complexe du projet. Initialement, le serveur tournait dans l'activité principale, ce qui signifiait qu'il s'arrêtait dès que l'application était fermée. Pour créer un vrai serveur autonome, j'ai refactorisé toute la logique réseau dans un Service Android.

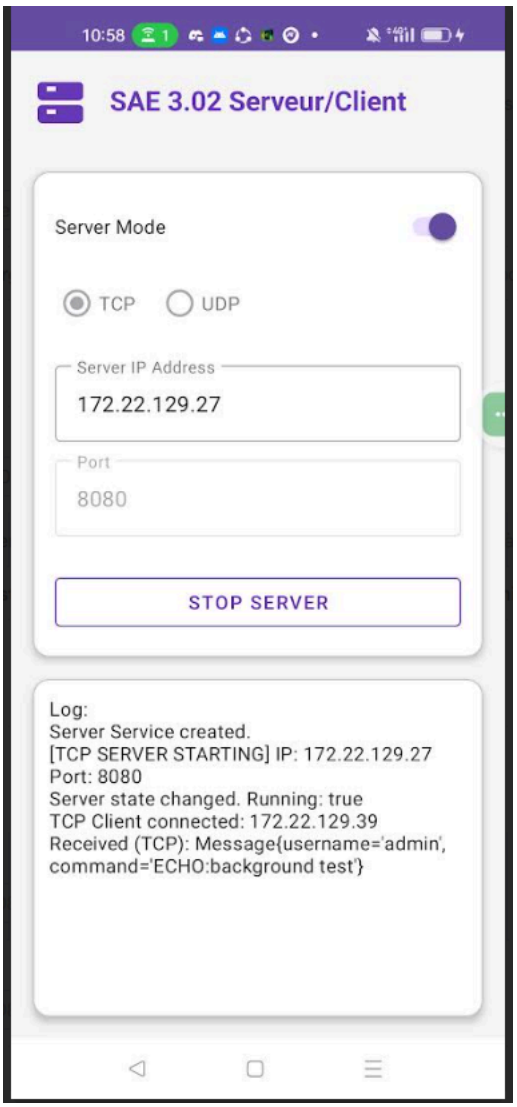
Démarche :

1. J'ai créé une nouvelle classe ServerService.java qui hérite de android.app.Service. Toute la gestion des Socket, des threads et du traitement des requêtes a été déplacée dans cette classe.

2. La MainActivity a été transformée en une simple "télécommande". Elle ne fait plus que lancer ou arrêter le service via des Intent.
3. Un BroadcastReceiver a été mis en place dans l'activité pour recevoir les messages (logs, changements d'état) envoyés par le service, permettant à l'interface de se mettre à jour en temps réel.

```
// Extrait de MainActivity.java, montrant comment le service est démarré
private void startServer() {
    Intent intent = new Intent(this, ServerService.class);
    intent.setAction(ServerService.ACTION_START_SERVER);
    intent.putExtra(ServerService.EXTRA_PORT, Integer.parseInt(editTextPort.getText().toString()));
    intent.putExtra(ServerService.EXTRA_PROTOCOL, currentProtocol);
    startService(intent);
}
```

Ce choix d'architecture a été validé par un test spécifique. La capture ci-dessous montre la preuve ultime : le serveur a été démarré, puis l'application a été mise en arrière-plan. Un client a ensuite envoyé une requête (ECHO:background test). Le log du serveur, consulté après avoir rouvert l'application, prouve que la requête a été reçue et traitée pendant que l'interface était inactive. Le serveur est donc bien autonome.


10:58
SAE 3.02 Serveur/Client
Server Mode
TCP
UDP
Server IP Address
172.22.129.27
Port
8080
STOP SERVER
Log:
Server Service created.
[TCP SERVER STARTING] IP: 172.22.129.27
Port: 8080
Server state changed. Running: true
TCP Client connected: 172.22.129.39
Received (TCP): Message{username='admin',
command='ECHO:background test'}

Conclusion

Ce projet a été une expérience formatrice qui m'a permis de mettre en pratique des concepts théoriques de programmation réseau dans un cadre concret et exigeant. L'objectif initial, qui était de développer une application client/serveur Android complète, a été non seulement atteint, mais aussi dépassé grâce à une série d'améliorations et à une démarche de développement rigoureuse.
Au cours de ce projet, j'ai pu implémenter et valider toutes les fonctionnalités du cahier des charges :

- La gestion des protocoles TCP et UDP, avec leurs spécificités propres.
- Le transport de données sérialisées pour des communications complexes.
- Un serveur multithread capable de gérer plusieurs clients simultanément.
- Une architecture multi-services flexible et évolutive.
- Un système de droits Qpour sécuriser les communications.

Le défi le plus important n'a pas été le codage des fonctionnalités elles-mêmes, mais la phase de débogage et de test. La résolution de problèmes complexes, comme le timeout en UDP lié à l'environnement de test des émulateurs ou les crashs silencieux de certains threads, m'a appris l'importance d'une méthodologie de test itérative et rigoureuse. C'est en isolant les problèmes pas à pas que j'ai pu rendre l'application véritablement robuste.
Les initiatives personnelles, notamment la refonte complète de l'interface graphique et la migration de la logique serveur vers un Service Android fonctionnant en arrière-plan, ont été les étapes qui ont apporté le plus de valeur au projet. Elles ont transformé une simple preuve de concept en une application solide, soignée et professionnelle.

Au final, ce projet m'a apporté une compréhension bien plus profonde du cycle de vie des applications Android, des subtilités de

la programmation réseau en Java et de l'importance de construire une architecture logicielle propre et maintenable.

Annexe dépôt du code

1. Introduction à la Gestion de Version

J'ai mis à disposition le code source complet de ce projet sur GitHub pour en faciliter la consultation. L'utilisation de Git en parallèle m'a permis de sauvegarder et d'organiser le développement à chaque étape clé, dont les détails sont présentés dans ce rapport.

2. Accéder au Projet sur GitHub

Le projet est accessible publiquement à l'adresse suivante :

https://github.com/Maxence0587416/SAE302-Client-Serveur